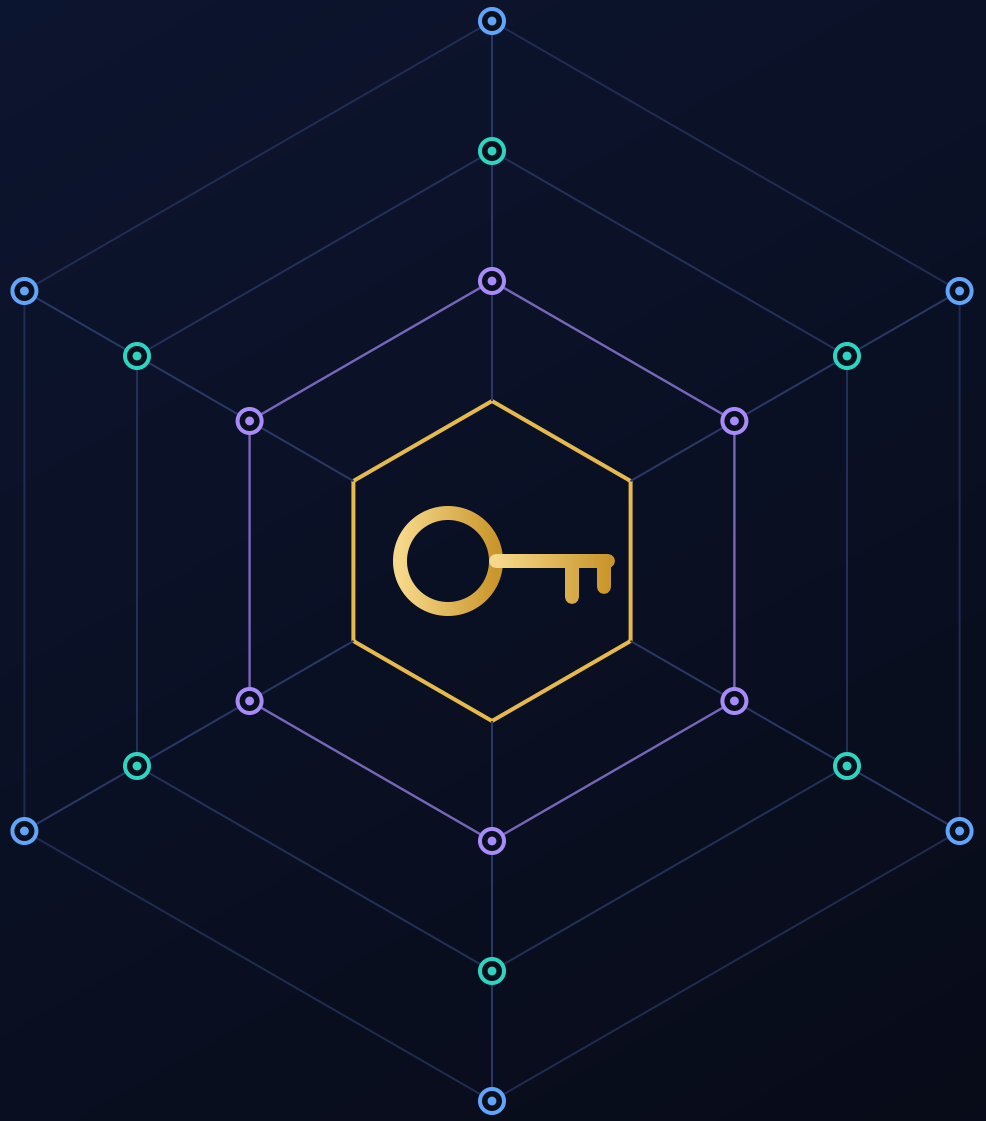
DEVELOPMENT-TEAM VISUAL REFERENCE

HUSHHAWK

Visual System Architecture

The A-to-Z blueprint, drawn step by step — from raw source evidence to CrownKey, graph, CRM and production release.

PostgreSQL · CrownKey · Neo4j · Ingestion · CRM · API · Security · QA · Deployment



29 diagrams · 31 pages · one connected story

Version 1.0 | 22 October 2026 | Derived from the HushHawk A-to-Z System Blueprint

CONTENTS

Your guided journey

Twenty-nine steps, eight chapters — follow in order, or jump to any chapter.

A Foundation

1 System at a glance p.03
2 Layer responsibilities p.04
3 Technology stack p.05
4 Domain model p.06
5 PostgreSQL schemas p.07

B Data engine

6 Connector lifecycle p.08
7 Incremental data & outages p.09
8 Identity resolution ladder p.10
9 Human review loop p.11

C Identity & keys

10 CrownKey anatomy p.12
11 Atomic event counter p.13
12 Pillar evidence rules p.14

D Graph & services

13 Neo4j graph model p.15
14 Outbox → graph sync p.16
15 API service map p.17
16 Frontend modules & roles p.18

E Business layer

17 CRM & Podio coexistence p.19
18 Attribution & pipeline p.20
19 Outbound compliance gate p.21

F Trust & operations

20 Security & ownership p.22
21 Observability p.23
22 Test plan & connector DoD p.24

G Delivery

23 Phases & exit gates p.25
24 First 30 build tasks p.26
25 Repository & workflow p.27

H Proof & governance

26 End-to-end example p.28
27 Non-negotiable rules p.29
28 Production release gate p.30
29 Client decisions p.31

HOW TO READ THE LABELS

SOURCE REQUIREMENT — Stated explicitly in the master guide

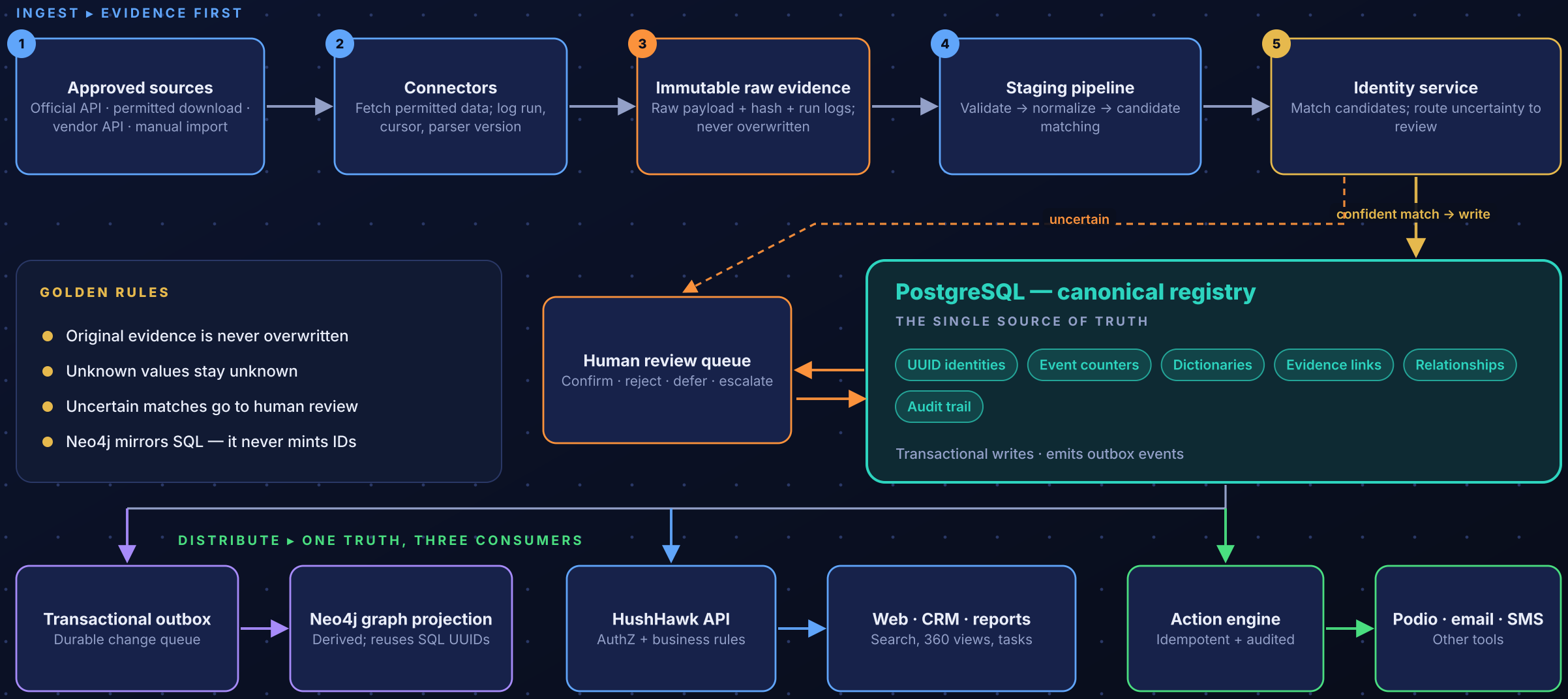
RECOMMENDATION — Engineering approach proposed

OWNER APPROVAL — Business decision — never guess

CHAPTER A · FOUNDATION

1 System at a glance

Everything HushHawk does, on one canvas: evidence in, one canonical truth, three controlled outputs.



Flow follows the master guide: connectors → immutable raw records → staging → PostgreSQL (↔ review) → outbox / API / action engine.

2

CHAPTER A · FOUNDATION

Layer responsibilities

Eight layers, strict boundaries: what each one owns — and what it must never do.

LAYER	OWNS ✓	MUST NOT DO ✕
Connector	Fetch permitted source data; log run, cursor, parser version	Bypass terms/rate limits or silently drop errors
Raw / staging	Original payload and transformed staging data	Overwrite original evidence
PostgreSQL	Canonical UUIDs, counters, dictionaries, evidence, audit	Delegate canonical identity to Neo4j
Identity service	Match candidates and route uncertainty to review	Merge weak matches automatically
Neo4j	Evidence-backed relationship traversal	Mint competing IDs or counters
API	Authorization and business rules	Expose database secrets to browser
UI / CRM	Search, profiles, lead work and tasks	Call vendor systems directly from frontend
Action engine	Approved external actions with idempotency / audit	Uncontrolled bulk outreach

CHAPTER A · FOUNDATION

3

Technology stack

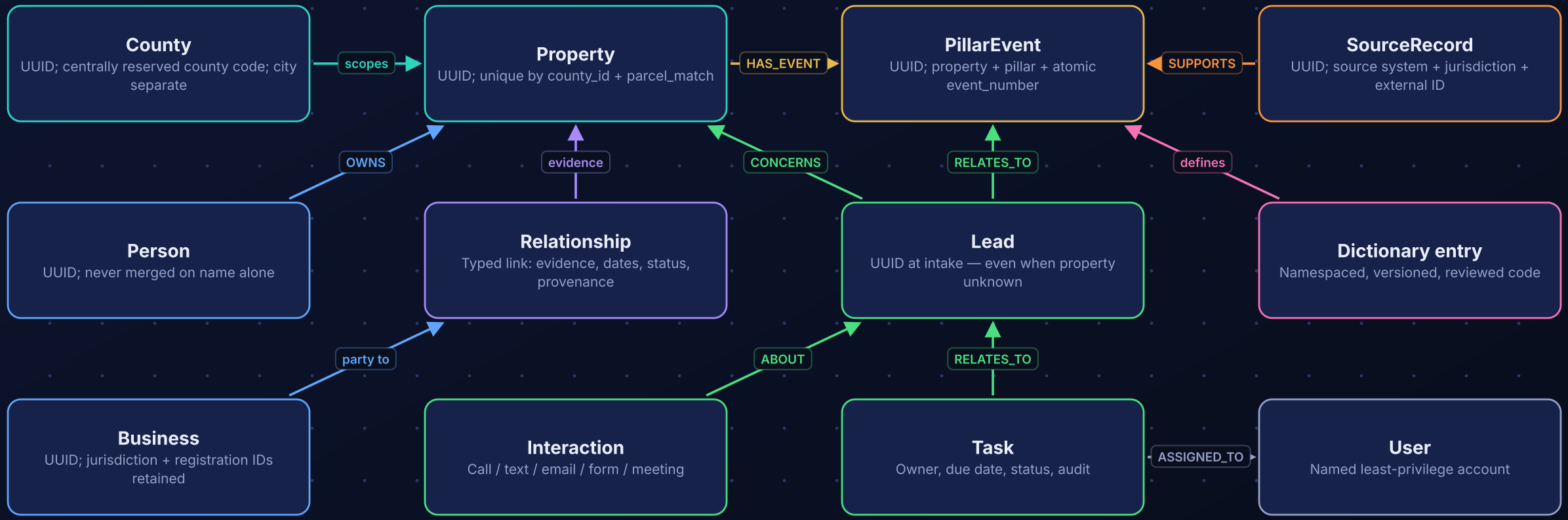
Four tiers, each with one clear job — and a decision note for every choice.



Client-owned infrastructure throughout — Docker, CI/CD, structured logs, metrics and alerts keep every tier observable and recoverable.

Domain model

Twelve entities, one identity rule each — joined by typed, evidence-backed relationships.



Audit event — append-only trail across every entity

Records who changed what, when, from which request — with a redacted before/after diff. Every identity, event and review decision above is covered.

actor · action · entity · timestamp · request ID · redacted diff

5

CHAPTER A · FOUNDATION

PostgreSQL schema map

Seven namespaces keep evidence, identity, relationships, CRM, machinery and audit cleanly apart.

ref — REFERENCE DATA

- ref.counties
- ref.pillars
- ref.lead_sources
- ref.platforms
- ref.contact_methods
- ref.person_roles
- ref.dictionary_versions
- ref.code_aliases

Counties, pillars, source codes, platforms, methods, roles, dictionary versions and aliases.

raw — ORIGINAL EVIDENCE

- raw.connector_runs
- raw.source_records
- raw.source_record_versions
- raw.attachments

Raw source payloads, source-record versions, connector runs and attachments.

core — CANONICAL ENTITIES

- core.people
- core.businesses
- core.properties
- core.property_aliases
- core.leads
- core.pillar_events
- core.campaigns
- core.interactions
- core.tasks
- core.users

rel — CONNECTIONS & DECISIONS

- rel.entity_relationships
- rel.identity_candidates
- rel.review_decisions
- rel.event_evidence

Relationships, match candidates, evidence links and review decisions.

crm — PIPELINE & SYNC

- crm.pipeline_stages
- crm.lead_stage_history
- crm.suppression_records
- crm.external_sync_map

ops — MACHINERY

- ops.event_counters
- ops.outbox_events
- ops.job_runs
- ops.job_errors
- ops.review_queue

audit — APPEND-ONLY

- audit.audit_events

Append-only security and data-change events: who, what, when.

KEY CONSTRAINTS

- parcel_raw is TEXT — never numeric
- UNIQUE(county_id, parcel_match)
- UNIQUE(property_id, pillar_code, event_number)
- UNIQUE(event_id, source_record_id)
- production_enabled defaults FALSE

6

CHAPTER B · DATA ENGINE

Connector lifecycle

The ten-step standard run every connector follows — from approval to alerting.

PREPARE

1 Verify approval

Source approved, permitted use confirmed, credentials and rate limit in place.

2 Open the run

Create connector_run: run ID, start time, parser version and cursor.

3 Fetch

Pull records only via a documented API, download or approved workflow.

PRESERVE

4 Store raw first

Save the raw payload and metadata BEFORE transformation; compute the payload hash.

5 Validate

Check required fields. Quarantine malformed records — never silently drop them.

RESOLVE

6 Normalize

Transform into staging without changing the raw evidence.

7 Match candidates

Apply approved rules; uncertain matches are routed to human review.

8 Write canonical

Commit canonical changes transactionally and emit outbox events.

OBSERVE

9 Record the run

Fetched / parsed / rejected / changed counts, errors, retries, final status.

10 Alert

Repeated failures, unexpected zero rows, parser drift, stale source data.

PRINCIPLE Raw evidence is stored before transformation and never altered by normalization. A failed run must never look successful.

Incremental data & source outages

The registry decides which sources may run; the strategy decides how — and what absence really means.

CONNECTOR REGISTRY — ONE ROW PER SOURCE

`connector_id / source_system`	Stable source name and internal connector ID
`jurisdiction`	County / state scope
`access_type`	Official API, permitted download/scraping, vendor API or manual import
`credentials_owner`	Client account owner and access contact
`permitted_use / terms_reviewed_at`	Approved use and review timestamp
`rate_limit / schedule`	Allowed request rate and refresh cadence
`cursor / watermark`	Incremental resume point
`parser_version`	Version of extraction / normalization code
`last_success / last_error`	Health state
`raw_storage_policy`	Retention, encryption and access
`approval_status / enabled`	Only approved sources may run in production

INCREMENTAL STRATEGY & OUTAGE LOGIC



Failed runs must never appear successful — log, retry safely, alert.

8

CHAPTER B · DATA ENGINE

Identity resolution ladder

Six rungs from certain to contradictory — the weaker the signal, the more human the decision.

STAGE	RULE	CONFIDENCE	OUTCOME
1 Exact external ID	Same source + jurisdiction + stable source ID	very high	Update the same source record
2 Exact parcel	Approved county + full normalized parcel	high	Candidate property match
3 Exact event	Jurisdiction-scoped case / instrument / account / year	high	Update same event; add evidence
4 Strong multi-field	Several independent identifiers agree	medium-high	Auto-link only under approved rule / threshold
5 Fuzzy candidate	Name / address similarity or incomplete identifiers	low	Review queue — no automatic merge
6 Conflict	Strong identifiers disagree	contradictory	Quarantine / escalate; preserve both records

HARD RULE Never merge people solely on shared name, phone or mailing address — always keep relationship evidence and verification state.

Human review loop

When the machine is unsure, a person decides — with full context, a recorded reason, and a way back.

WHAT ENTERS THE QUEUE

- Duplicate candidates
- Conflicting evidence
- Parser errors

REVIEW SCREEN SHOWS

1. Raw and normalized values side by side
2. Source links
3. Observed dates
4. Matching rules that fired
5. Conflicts between records

REVIEWER ACTIONS

Confirm
Evidence supports the match

Reject
Not the same entity

Defer
Need more evidence

Link without merging
Create a relationship; keep both identities

Escalate
Pass to a senior owner

DECISION RECORD

`reviewer`	who decided
`timestamp`	when
`reason`	why
`decision`	what was chosen

Full history preserved — nothing is overwritten.

Controlled reversal
Any decision can be reversed through a controlled, audited process.

Weak evidence never merges on its own: it waits here for a named human, a reason and an audit entry.

10

CHAPTER C · IDENTITY & KEYS

CrownKey anatomy

A human-readable label assembled from structured fields — the UUID stays the real identity.

County code
Centrally reserved; city kept separate

Pillar code
From the approved dictionary

First source code
Shown only when the source is known

DUV-[PARCEL]-FC-001 · MAIL

Parcel
parcel_raw stored as TEXT: zeros & punctuation kept

Event number
Per property + pillar; atomic; never reused

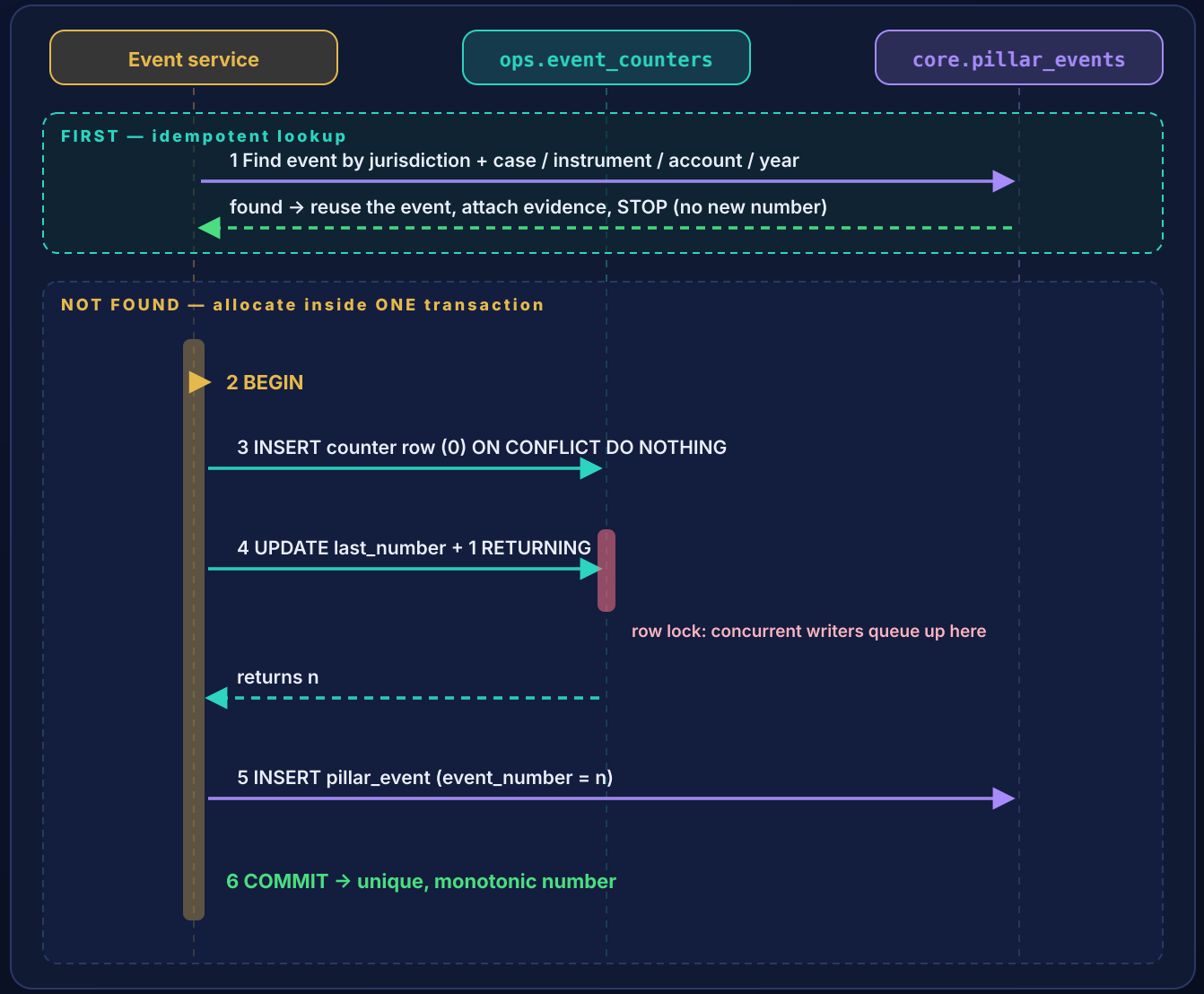
FIVE DISPLAY CASES

CASE	PATTERN	EXAMPLE ONLY
Property known, no event	`COUNTY-PARCEL`	`DUV-[PARCEL]`
Source known, no event	`COUNTY-PARCEL · SOURCE`	`DUV-[PARCEL] · MAIL`
Event and source known	`COUNTY-PARCEL-PILLAR-NNN · SOURCE`	`DUV-[PARCEL]-FC-001 · MAIL`
Event, source unknown	`COUNTY-PARCEL-PILLAR-NNN`	`DUV-[PARCEL]-FC-001`
Parcel unknown	`Property pending (lead still has lead_id)`	`Property pending`

DISPLAY ONLY Never a primary key, foreign key or API join key · never parse by splitting on hyphens · always render from structured fields.

Atomic event counter

Concurrent writers can never produce a duplicate or skipped event number.



WHY NEVER MAX(event_number) + 1

UNPROTECTED

Worker A	read MAX = 3	insert 4
Worker B	read MAX = 3	insert 4

✕ Two events both numbered 4 → collision, lost update or error

ATOMIC COUNTER

Worker A	UPDATE → 4	commit
Worker B	UPDATE waits...	UPDATE → 5

✓ Serialised by the row lock → 4 then 5, never a duplicate

per property + pillar · starts at 1 · never reused

The SQL shown is a pattern; production adds error handling, permissions and transaction tests.

Pillar evidence rules

Every pillar needs proof and a definition of “distinct” before it can create events.

FC — Foreclosure

MINIMUM EVIDENCE
Clerk / court case and current case status

DISTINCT EVENT RULE
Jurisdiction + full case reference

TAX — Tax delinquency

MINIMUM EVIDENCE
Balance, tax year and checked date

DISTINCT EVENT RULE
Tax account + tax year

ML — Municipal lien

MINIMUM EVIDENCE
Recorded lien and release / satisfaction search

DISTINCT EVENT RULE
Recording jurisdiction + lien instrument

PRB — Probate

MINIMUM EVIDENCE
Probate record and evidenced property link

DISTINCT EVENT RULE
Jurisdiction + case reference

VAC — Vacancy

MINIMUM EVIDENCE
Dated, attributable observation or reliable record

DISTINCT EVENT RULE
Approved episode definition

Other — Any new code

MINIMUM EVIDENCE
Must define evidence and distinct-event rules first

DISTINCT EVENT RULE
Owner-approved per pillar

OVERLAPPING PILLARS — RELATED, NOT DOUBLE-COUNTED

ABS ↔ OOS

TAX ↔ TCERT ↔ TDEED

CODE ↔ CONDEMN

FC ↔ PFC

They can be linked as related events, but are not counted as separate unique leads by default.

TRACK TWO STATES — SEPARATELY

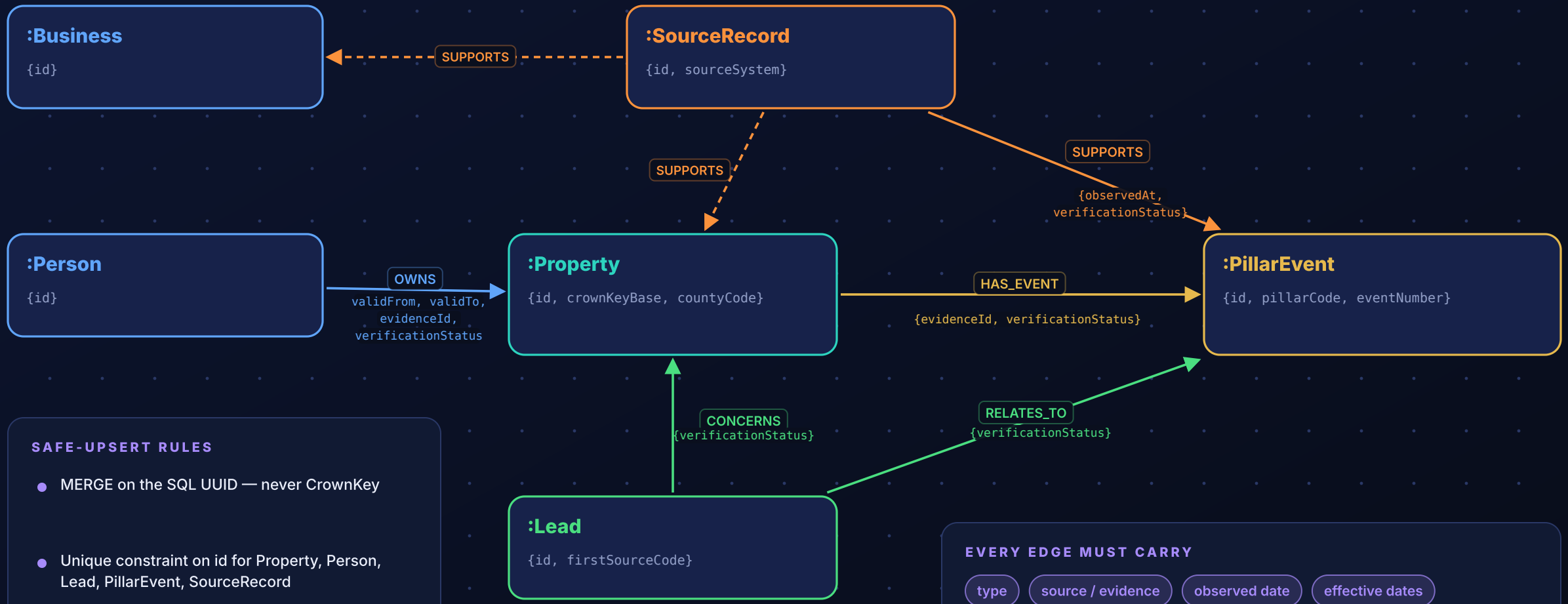
Event status: OPEN · RESOLVED · HISTORICAL · UNKNOWN

Verification: CONFIRMED · POSSIBLE · NEEDS_REVIEW · REJECTED

Lifecycle (is it open?) and trust (is it proven?) answer different questions — never collapse them into one field.

Neo4j graph model

A derived view for traversal: every node keyed by the SQL UUID, every edge carrying its evidence.



SAFE-UPSERT RULES

- MERGE on the SQL UUID — never CrownKey
- Unique constraint on id for Property, Person, Lead, PillarEvent, SourceRecord
- Idempotent upserts; no competing IDs or counters

EVERY EDGE MUST CARRY

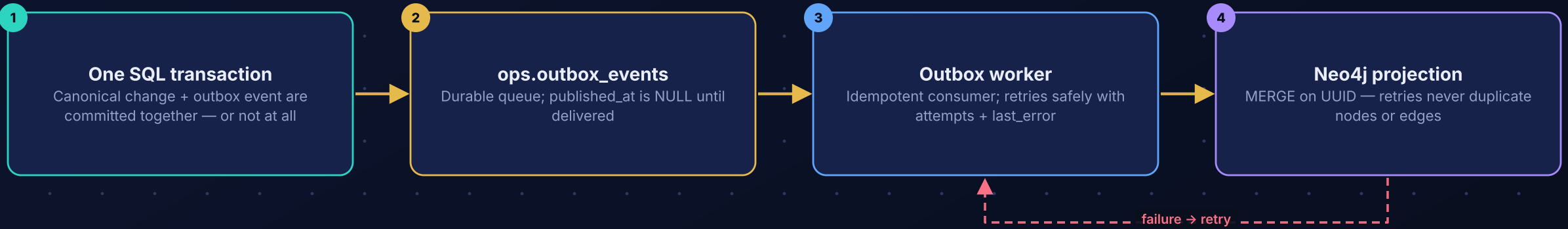
type · source / evidence · observed date · effective dates · verification status · review / confidence

Traversals apply authorization filters and depth / row limits.

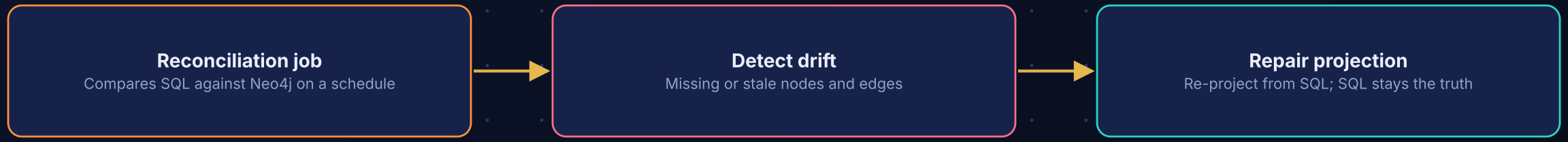
Outbox → graph sync

A transactional outbox keeps SQL and Neo4j in step — with retries, reconciliation and visible lag.

PROJECT ▸ SQL → GRAPH, NEVER THE REVERSE



RECONCILE ▸ PROVE THE TWO STORES AGREE



WATCH ▸ WHAT OPERATIONS SEES

Outbox backlog
Pending events waiting to publish

Graph sync lag
Time between SQL commit and graph update

Retry attempts
Repeated failures surface as alerts

Read-side limits
Authorization filters + depth / row limits

Neo4j is added after the edge model and SQL rules stabilize, and never issues IDs or event counters.

15

CHAPTER D · GRAPH & SERVICES

API service map

Ten services behind one versioned, authorizing front door.

Web workspace · CRM · reports — browser never holds database secrets

↓

HushHawk API /api/v1 · OpenAPI-documented · server-side authorization on every request

Identity
Create / resolve entities; match candidates; review decisions

CrownKey
Render references; resolve aliases; allocate counters via SQL transaction

Source
Connector registry, source records, runs, permissioned raw access

Event
Create / update pillar events, attach evidence, update status with audit

Lead / CRM
Intake, attribution, pipeline, assignment, interactions, tasks

Graph
Authorized relationship traversal and graph views

Search
Universal search, filters, pagination, provenance

Admin
Dictionary versions, code activation, roles, connectors, feature flags

Action
Queue external actions, suppression checks, retries, audit

Reporting
Unique-lead metrics, event counts, attribution, source health, quality

RULES FOR EVERY ENDPOINT

- Version contracts and document with OpenAPI
- Authorize on the server — hiding a button is not security
- Idempotency keys on retryable create / action calls
- Pagination and query timeouts for large searches
- Never send DB credentials or unrestricted raw PII to the browser
- Return provenance and verification state with derived results
- Correlation IDs across API, workers, outbox, graph sync, audit

Frontend modules & roles

What people can see and do — one workspace, six role profiles, server-enforced.

ELEVEN WORKSPACE MODULES

Login / access
Role-aware routes, MFA / SSO if available, session control

Global search
Person, business, property, parcel, CrownKey, case ID

Property 360
Identity, aliases, ownership evidence, events, source history, timeline, tasks

Person 360
Verified identity / contact data, roles, relationships, properties, interactions

Business profile
Legal identifiers, related people and properties, source evidence

Lead workspace
Source, stage, assignment, next task, interactions, linked property / event

Relationship graph
Typed edges, provenance, verification, filters, bounded depth

Review queue
Duplicate candidates, conflicting evidence, parser errors, decisions

Connector health
Last run, counts, failures, stale sources, retries, alerts

Admin
Dictionary approval / versioning, users, connectors, audit, feature flags

Reports
Unique leads vs pillar memberships; first source vs latest touch; quality

Typed API client — modules never call vendor systems directly.

Every screen shows provenance and verification state, and every action is authorized again on the server.

SUGGESTED ROLES — LEAST PRIVILEGE

Super Admin
User / role administration and audited production configuration

Manager
Assignments, reviews, pipeline and operational reports

Acquisitions
Authorized leads, property details, interactions, follow-up

Marketing
Campaigns, attribution, approved lists, campaign reporting

Data Reviewer
Resolve candidate matches; no global dictionary activation by default

Developer / Ops
System health and technical logs; raw personal data only when required

OWNER APPROVAL — client approves final roles

17

CHAPTER E · BUSINESS LAYER

CRM & Podio coexistence

Build the custom CRM on HushHawk's model; keep Podio connected until parity is proven and signed off.

SYSTEM OF RECORD

HushHawk

Canonical data model + custom CRM: leads, contacts, properties, acquisitions, dispositions, stages, tasks, follow-ups, communications, users, permissions, activity history

crm.external_sync_map

internal UUID ↔ Podio app / item ID · last sync · hash / version · status

TRANSITION SYSTEM

Podio

Stays connected as a transition / integration system until parity and migration are approved

- Never use Podio IDs as HushHawk primary keys
- One system of record declared per field
- Idempotent sync · conflict handling · retry queue
- Sync-health dashboard visible to admins

COEXISTENCE ROADMAP ▸ NO CUT-OVER WITHOUT SIGN-OFF

1. **HushHawk owns the model** — From day one
2. **Podio stays connected** — Mapped through sync map
3. **Shadow comparison** — Records, attachments, tasks, stages, history
4. **Parity + migration approved** — Client sign-off gate
5. **Retire Podio** — Only after sign-off

ROLLBACK + EXPORT PLAN kept ready at every stage — Podio is not retired until the client signs off.

RECOMMENDED

HushHawk owns the canonical data model and the custom CRM experience from the beginning; Podio is an integration, not the foundation. Run shadow comparisons before any cut-over and reconcile every record, attachment, task, stage and history item.

18

CHAPTER E · BUSINESS LAYER

Attribution & pipeline

One lead, one original source — everything after is history, not a new lead.

ATTRIBUTION ▸ THE FIRST SOURCE NEVER CHANGES

LOCKED — original known source

Lead created
first_source_code = MAIL (example)

Later touches are INTERACTIONS — they do not create new leads

Call · Text · Email · Form · Visit

STORED SEPARATELY WHEN KNOWN

platform · medium · placement · campaign · UTM / click / tracking no. · referrer

PIPELINE ▸ SUGGESTED LIFECYCLE (FINALIZE WITH CLIENT)

NEW → CONTACTED → QUALIFIED → OFFER → CONTRACT → CLOSED

FOLLOW_UP / LOST
with reason and audit trail

Count correctly

Unique leads are counted separately from pillar memberships; first source is reported separately from latest touch.

Unknown means unknown

OTHER requires source_detail. Never guess a source or a platform.

Imports are not leads

A source-data import is not automatically an inbound lead or proof of ownership.

Scores explain themselves

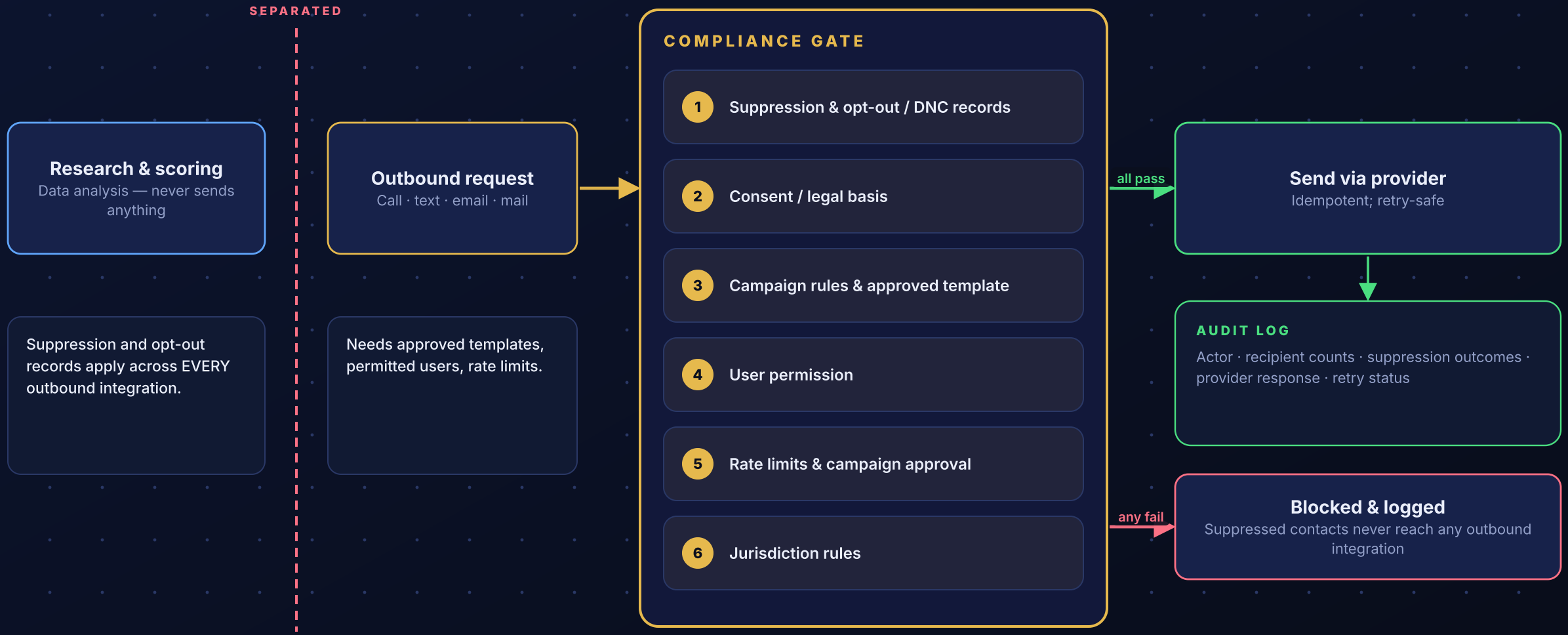
Priority scores need an approved, versioned formula; a high score never turns unverified evidence into fact.

19

CHAPTER E · BUSINESS LAYER

Outbound compliance gate

Research and scoring are separate from sending — and nothing leaves without passing the gate.



LOCKED Mass outreach stays off until identity, suppression, duplicate, audit, monitoring and recovery controls pass. Obtain legal review — this is not legal advice.

20

CHAPTER F · TRUST & OPERATIONS

Security & ownership

The client owns every key asset; access is named, minimal, separated and proven recoverable.

THE CLIENT COMPANY OWNS

- GitHub organization
- Cloud account
- Production database
- Object storage
- Domains
- Vendor accounts

THREE ENVIRONMENTS — NEVER SHARING CREDENTIALS OR DATA ACCESS

Development
Own credentials

→

Staging
Synthetic or approved test data; release rehearsal first

→

Production
Named least-privilege access; audited changes

EIGHT CONTROLS

Named accounts
Least privilege — no shared personal logins

Secrets manager
Never commit secrets or share production credentials in chat; rotate tokens

Encrypt everywhere
In transit and at rest

Data lifecycle
Retention, deletion and backup policies defined

Restrict sensitive data
Sensitive fields and raw source payloads locked down

Audit what matters
Important reads, exports, merges and admin changes

Separate duties
Technical administration apart from business data access

Prove recovery
Test restore, incident response, credential revocation, deployment rollback

21

CHAPTER F · TRUST & OPERATIONS

Observability

If a job runs, we can see it; if it fails, we know; if it drifts, we are alerted.

EIGHT SIGNALS WE WATCH

Connector success

MEASURES
Last successful run per source

PURPOSE
Detect outages & parser failures

Run counts

MEASURES
Fetched / parsed / rejected / changed

PURPOSE
Detect empty results & quality regressions

Review backlog

MEASURES
Queue size and candidate rate

PURPOSE
Monitor identity matching

Outbox & graph lag

MEASURES
Pending events and sync delay

PURPOSE
Detect a stale graph projection

API health

MEASURES
Latency and error rate

PURPOSE
Workspace health

Database capacity

MEASURES
Storage, CPU, connections

PURPOSE
Capacity and reliability

Action failures

MEASURES
Failures and retries

PURPOSE
Prevent silent task or CRM loss

Access patterns

MEASURES
Unusual exports or reads

PURPOSE
Detect misuse

OPERATING PRINCIPLES

Observable

Every job emits state that authorized admins can see.

Recoverable

Failed runs retry safely and idempotently.

Honest

A failed run must never appear successful.

Traceable

Structured logs and correlation IDs end to end.

22

CHAPTER F · TRUST & OPERATIONS

Test plan & connector DoD

Twelve platform-level proofs and a seven-point checklist every connector must clear.

ACCEPTANCE TESTS — THE PLATFORM MUST PASS ALL

Keys & parcels

- [x] CrownKey variants: base, event + source, event without source, property-pending lead
- [x] Leading-zero and punctuated parcels are preserved
- [x] Same parcel text in two counties stays two property identities

Events & counters

- [x] Concurrent event creation yields unique, monotonic numbers per property + pillar
- [x] Same event from two sources keeps one identity and adds evidence
- [x] Different external event references create distinct events

Ingestion & identity

- [x] Repeated imports are idempotent and keep observation history
- [x] Shared name / phone / address never auto-merges people
- [x] First source survives later interactions; corrections are audited

Graph, security & recovery

- [x] Neo4j retries safely; reconciliation repairs missing or stale projections
- [x] Direct API tests cover raw payloads, exports and admin actions
- [x] Suppressed contacts are excluded from outbound actions
- [x] Backup restore succeeds in a separate environment

CONNECTOR DEFINITION OF DONE

1. Access and permitted use documented and approved
2. Stable source IDs, jurisdiction, rate limits, retries, health metrics
3. Raw payload and parser version retained under policy
4. Field mapping and normalization documented
5. Idempotency and duplicate behavior tested
6. Uncertain matches route to review
7. Quality report and sample records signed off

Phases & exit gates

Five phases; each one closes only when its gate is passed.

PHASE 0 — Discovery & governance	PHASE 1 — Foundation	PHASE 2 — Data engine & identity	PHASE 3 — Relationship & search	PHASE 4 — Actions & production
SCOPE Scope, access, dictionaries, roles, ownership, acceptance tests	**SCOPE** Repo / CI, environments, PostgreSQL, reference tables, audit, CrownKey	**SCOPE** Raw / staging, first approved connectors, normalization, matching / review, outbox	**SCOPE** Neo4j projection if justified, universal search, timeline, graph, workspace	**SCOPE** CRM actions, approved communications, Podio transition, backup / monitoring
⚑ EXIT GATE Approved dictionary, source shortlist and system-of-record decisions	**⚑ EXIT GATE** Migrations and tests pass; placeholder records excluded; dictionary status clear	**⚑ EXIT GATE** Agreed P0 sources end to end with provenance, retry and quality tests	**⚑ EXIT GATE** Evidence and status visible; permissions and performance pass	**⚑ EXIT GATE** Security, suppression, audit, restore and user-acceptance sign-off

Exact dates, source count and deliverables follow the signed statement of work — no county or source is promised until access, terms, formats, limits and data quality are verified.

First 30 build tasks

The thirty concrete tasks that turn the blueprint into a working platform, grouped by phase.

PHASE 0 — Discovery & governance

1. Recover & version the legacy pillar dictionary
2. Approve counties & county-code reservation
3. Approve P0 sources, access, usage rights, schedules
4. Approve entity & relationship types
5. Decide Podio field-level source of truth

PHASE 1 — Foundation

6. Client-owned GitHub org & repositories
7. Dev / staging environments + secrets management
8. PostgreSQL migration & backup pipeline
9. Versioned reference dictionaries
10. UUIDs for property, person, business, lead, event
11. Parcel normalization with county tests
12. CrownKey renderer & validation
13. Event identity lookup + atomic counters

PHASE 2 — Data engine & identity

14. Raw source records + payload hashing
15. Connector registry + run logs
16. First approved connector in staging
17. Validation, quarantine, error reporting
18. Match candidates + review queue
19. Evidence links + verification status
20. First-source attribution + interactions
21. Outbox + idempotent consumer

PHASE 3 — Relationship & search

22. Neo4j constraints + one SQL→graph projection
23. Graph reconciliation report
24. API authentication + role checks
25. Global search + Property 360
26. Lead / task / interaction CRM basics

PHASE 4 — Actions & production

27. Connector health + data-quality dashboards
28. Acceptance tests on synthetic / approved data
29. Security, restore and rollback tests
30. Client sign-off before production enablement

Phase grouping is a suggested mapping of the thirty build tasks onto the five phases; final order follows the signed statement of work.

Repository & workflow

One monorepo layout, one disciplined path from pull request to production.

```
hushhawk/
├ apps/web/                  React / Next.js workspace
├ apps/api/                  Primary API
├ apps/worker-ingestion/     Connectors / parsers
├ apps/worker-outbox/        Graph / search / external sync
├ packages/contracts/        Shared API schemas
├ packages/crownkey/         Display renderer / validation
├ packages/identity/         Matching rules
├ database/migrations/       Immutable migrations
├ database/seeds/            Approved dictionaries only
├ graph/                     Constraints, projections, reconciliation
├ infrastructure/            Docker, CI, IaC if approved
├ docs/                      architecture · data-dictionary · source-connectors · runbooks · decisions
└ tests/                     unit · integration · e2e
```

DEVELOPER WORKFLOW

1. **Protected main + pull request**
   Review required; no direct pushes
2. **CI pipeline**
   Lint · type-check · unit + integration tests · security scan · migration validation
3. **Deploy to staging first**
   Synthetic or approved test data only
4. **Feature flags**
   New connectors, dictionary codes, graph features, outbound actions
5. **Production release**
   Expand / contract migrations · health checks · rollback · release notes
6. **Record the decision (ADR)**
   Context, choice, alternatives, consequences, approver

End-to-end example

One foreclosure, reported by two feeds, becomes one event with two pieces of evidence.



IF RECORDS CONFLICT

Route to the review queue — never silently overwrite. Both source records remain preserved.

Placeholders such as [PARCEL] are illustrative and must never be treated as real data.

27

CHAPTER H · PROOF & GOVERNANCE

Non-negotiable engineering rules

Thirteen things the platform must never do — each one a guardrail for identity, evidence or compliance.

NEVER #01

Use CrownKey as a primary key, foreign key or API join key

NEVER #02

Parse CrownKey by splitting on hyphens

NEVER #03

Store parcel identifiers as numeric types

NEVER #04

Use unprotected MAX(event_number) + 1

NEVER #05

Create a new event because a second source reported the same event

NEVER #06

Discard raw source evidence when normalized data changes

NEVER #07

Merge people solely by shared name, phone or mailing address

NEVER #08

Treat a source import as confirmed ownership or an inbound lead

NEVER #09

Count pillar memberships as unique leads

NEVER #10

Activate unapproved dictionary codes

NEVER #11

Let Neo4j issue canonical IDs or counters

NEVER #12

Run outbound actions without authorization, suppression checks, audit and retry safety

NEVER #13

Deploy without migrations, tests, monitoring and rollback

WHY THESE THIRTEEN

Together they protect identity (UUIDs stay canonical), evidence (raw data is never lost), counting (leads are unique) and compliance (outreach is authorized, audited and recoverable).

28

CHAPTER H · PROOF & GOVERNANCE

Production release gate

Nine technical gates, then one human signature.

DICTIONARY
Legacy reconciled; proposed entries disabled until approved

IDENTITY
UUIDs canonical; matching and alias history tested

CROWNKEY
Display tests pass; raw parcel formatting preserved

EVENTS
Atomic counters, idempotent updates and evidence tested

INGESTION
Raw payload, parser version, retries and health metrics present

GRAPH
SQL IDs reused; outbox and reconciliation tested

CRM
Attribution, interactions, tasks and sync ownership documented

SECURITY
RBAC, secrets, audit, backup and suppression tested

OPERATIONS
Monitoring, runbooks, restore test and rollback plan complete

★ **SIGN-OFF**
Client approves scope, source access, tests and production enablement

All nine gates must pass before the final sign-off — then, and only then, production sources and outbound actions are enabled.

29

CHAPTER H · PROOF & GOVERNANCE

Decisions the client must approve

Eleven business decisions that unlock the build — owned by the client, not assumed by developers.

1 OWNER APPROVAL

Complete legacy 29-pillar list and final status of every proposed code

2 OWNER APPROVAL

Initial counties and authoritative county codes

3 OWNER APPROVAL

P0 sources, access method, refresh cadence, terms and credentials owner

4 OWNER APPROVAL

Canonical person / property / business matching rules and review thresholds

5 OWNER APPROVAL

Distinct-event rules for each pillar, including release, closure and transitions

6 OWNER APPROVAL

Lead attribution rules, pipeline stages and scoring formula

7 OWNER APPROVAL

User roles and access to raw / sensitive fields

8 OWNER APPROVAL

Whether Neo4j is required in the first release or follows SQL validation

9 OWNER APPROVAL

Podio coexistence duration, field ownership, sync direction and migration gate

10 OWNER APPROVAL

Retention, privacy, suppression, exports, backup and incident response

11 OWNER APPROVAL

Acceptance tests, phase dates, budget and change-control process

DEVELOPERS MUST NOT GUESS

Each of these is a business decision. Until it is approved, the related feature stays disabled.